

Universal, Hard-Routed and Soft Mixture-of-Experts Image Restoration, and Style-Conditioned Face-to-Sketch Generation: Training, Evaluation and Deployment

Musa Khan

Generative AI – Assignment 1

Code: <https://github.com/MusaxKhan/Restoration-GAN>

Demo video: [YOUTUBE_LINK_TO_BE_ADDED](#)

Abstract—We design, train, evaluate and deploy four related generative systems. Tasks 1–3 restore 128×128 Oxford-IIIT Pet images corrupted at run time by salt-and-pepper noise, Gaussian blur or rectangular occlusion: (1) one universal denoising autoencoder with a genuine compressed latent code, (2) a corruption classifier that hard-routes images to three independently trained specialist autoencoders, and (3) a jointly fine-tuned soft mixture-of-experts whose gate is initialised from the classifier and whose experts come from Task 2. Task 4 is a style-conditioned conditional GAN (U-Net generator, PatchGAN discriminator) that turns FS2K face photographs into sketches in one of three styles. All hyper-parameters were tuned with Optuna, experiments are tracked in MLflow, every inference model is exported to ONNX and verified against PyTorch, and the four tasks are served by a single React/FastAPI web application that runs through Docker Compose. We report quantitative results, routing behaviour of the gate, failure cases and the application design that was prototyped in Google Stitch.

Index Terms—denoising autoencoder, mixture of experts, image restoration, conditional GAN, Optuna, ONNX, deployment

I. INTRODUCTION

Image restoration is rarely a single problem: real inputs may be clean, noisy, blurred or partly missing, and the type of damage is usually unknown. A single network must then compromise between very different inverse problems, whereas a pipeline of specialists needs a reliable way to decide which specialist to use. Tasks 1–3 of this assignment study exactly this trade-off on synthetic but controlled corruptions: a universal autoencoder (UAE), a classifier-driven hard routing system, and a differentiable soft mixture-of-experts (MoE) in which the gate and the experts are trained jointly. Task 4 addresses a different generative problem, paired photo-to-sketch translation with a conditional GAN, where the sketch style is an explicit categorical condition.

The assignment is as much an engineering exercise as a modelling one: all models must be tuned (Optuna), tracked (MLflow), exported (ONNX), served (FastAPI) behind a React interface and shipped as Docker containers. This report documents each task’s architecture, losses, training procedure and hyper-parameter search, presents the results obtained from

the final trained models, analyses failure cases, and describes the application.

II. RELATED WORK AND DESIGN DECISIONS

Denoising autoencoders. Denoising autoencoders learn robust representations by reconstructing a clean input from a corrupted one [1], and deep autoencoders with a low-dimensional bottleneck were shown to be effective non-linear dimensionality-reduction tools [2]. Skip connections between encoder and decoder (as in U-Net [3]) make restoration easy but allow the network to copy corrupted pixels; because Task 1 requires a meaningful bottleneck, we use *no* skip connections, so that every pixel of the output must pass through the latent vector. This choice trades fidelity for a true compressed representation, a trade-off we quantify in Section VI-A.

Losses. Pixel losses such as L_2 produce blurry output and are poorly correlated with perceived quality; Zhao et al. [4] showed that mixing L_1 with a structural-similarity term (SSIM [5]) converges better and gives better quality than either alone. We therefore use $\mathcal{L} = \alpha \mathcal{L}_1 + (1 - \alpha)(1 - \text{SSIM})$, with α selected by Optuna instead of fixed to 0.8.

All-in-one restoration. Recent work addresses unknown corruption types with one model, e.g. AirNet, which learns a degradation representation by contrastive learning and conditions the restoration network on it [6]. Our hard-routed and soft-MoE systems explore a simpler alternative: explicit corruption classification and expert selection.

Mixture of experts. Adaptive mixtures of local experts [7] combine specialists through a trainable gate; sparsely-gated MoE layers [8] and Switch Transformers [9] add auxiliary load-balancing losses to prevent routing collapse, where the gate sends nearly every input to one expert. Our balance term (Section IV-D) follows the same motivation, using a squared deviation of the batch-average gate weights from the uniform value $1/4$ on class-balanced batches, as suggested in the assignment.

Conditional GANs and sketches. Generative adversarial networks [10] and their conditional form [11] underlie pix2pix

[12], which pairs a U-Net generator with a PatchGAN discriminator [13] and adds an L_1 term to the adversarial loss. The FS2K dataset [14] provides 2,104 photograph–sketch pairs in three sketch styles; its authors’ FSGAN baseline also uses a style vector, which motivates our learned style embedding.

Tooling. Hyper-parameters are tuned with Optuna’s TPE sampler and median pruner [15], runs are tracked with MLflow [16], models are exported to ONNX and executed with ONNX Runtime [17], and the API uses FastAPI [18]. The interface was prototyped in Google Stitch [19].

III. DATASETS AND CORRUPTION PROTOCOL

A. Oxford-IIIT Pet (Tasks 1–3)

The Oxford-IIIT Pet dataset [20] provides 3,680 official train/validation images and 3,669 official test images of 37 breeds; labels are not used. All images are converted to RGB and resized to 128×128 . The official train/validation collection is split 80/20 with random seed 42 (2,944 / 736 images) and the *same* split is used in Tasks 1–3. The official test set is untouched until the final evaluation.

B. Run-time corruption

Corruptions are generated inside the data loader; no corrupted copies are stored. For every training image one of four conditions is drawn with equal probability and a fresh severity is sampled at every access: *clean*; *salt-and-pepper* with probability $p \sim \mathcal{U}(0.02, 0.15)$, affected pixels set to black or white with equal probability; *Gaussian blur* with kernel size in $\{3, 5, 7\}$ and $\sigma \sim \mathcal{U}(0.5, 2.5)$; *rectangular occlusion* with 1–3 non-overlapping black rectangles jointly covering 10–35% of the image at random positions (random aspect ratio in $[0.5, 2]$). A single function maps an image and a corruption *specification* to the corrupted image, so the same code serves training (specification sampled on the fly) and evaluation (specification read from a manifest).

C. Deterministic manifests

A validation manifest (one corruption per validation image, 736 entries, type uniform, severity from the training distribution) and a test manifest were generated once from fixed seeds and stored in JSON together with the corruption type, severity, rectangle coordinates, blur settings and the salt-and-pepper seed. The test manifest contains for every test image the clean image and three fixed severities per corruption: $p \in \{0.03, 0.08, 0.15\}$; blur $(k, \sigma) \in \{(3, 0.7), (5, 1.5), (7, 2.5)\}$; and 1/2/3 rectangles covering 10/20/35% (36,690 entries in total). Unit tests verify the split, the corruption statistics (e.g. measured coverage within 4% of target), determinism and that the training pipeline re-samples corruptions at every access. Example corruptions are shown in Fig. 1.

D. FS2K (Task 4)

FS2K contains 1,058 official training pairs and 1,046 official test pairs with three sketch styles (style index 0/1/2 is shown as Style 1/2/3). 15% of the training pairs (stratified by style, seed 42) form the validation set; the official test set is used only for

the final evaluation. Photographs and sketches are resized to 128×128 with their pairing preserved. Paired augmentation applies the *same* random horizontal flip and the *same* random crop box (scale 0.8–1.0, resized back) to both images; a unit test checks that augmentation preserves alignment.

IV. METHODS

A. Common training set-up

Models are implemented in PyTorch. The restoration networks use AdamW [21] with a one-cycle learning-rate schedule [22], gradient clipping at 5 and mixed precision on the GPU; GANs use Adam [23] with $\beta = (0.5, 0.999)$. Batch normalisation [24] is used in the convolutional blocks. The validation *objective* used by every Optuna study is independent of any tuned loss weight: for restoration it is the mean validation $L_1 + (1 - \text{SSIM})$ (lower is better); for the classifier 1–macro-F1; for the GAN the validation $L_1 + (1 - \text{SSIM})$ of generated against ground-truth sketches. SSIM is our own differentiable implementation (11×11 Gaussian window, $\sigma = 1.5$), unit-tested against scikit-image. PSNR is reported with a 50dB cap so that exact identity does not yield infinite values. All experiments are logged to MLflow (parameters, per-epoch curves, final checkpoints, sample images), and Optuna studies are stored in SQLite so that interrupted runs can resume.

B. Task 1: universal denoising autoencoder

Architecture. The encoder has four stride-2 convolutional stages ($128 \rightarrow 8$ pixels) whose channel count doubles from c to $8c$, each stage being a 4×4 stride-2 convolution followed by a 3×3 convolution (batch norm, LeakyReLU). The $8 \times 8 \times 8c$ feature map is flattened and projected by a linear layer to a latent vector of dimension d (dropout p), i.e. a compression of $49,152/d$ times; a linear layer re-expands it and four transposed-convolution stages and a final 3×3 convolution with a sigmoid reconstruct the 128×128 RGB image. There are no skip connections. The network is never told the corruption type.

Loss. $\mathcal{L} = \alpha L_1(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x}))$ with x the clean target and $\hat{x} = D(E(\tilde{x}))$ the reconstruction of the corrupted input $\tilde{x}$.

Optuna. Learning rate, batch size, latent dimension d (bottleneck), number of encoder channels c , dropout and α were searched (Table I); each trial trains for a reduced budget on a subset of the training split, with median pruning.

C. Task 2: classifier and hard-routed specialists

Classifier. A plain CNN with d_c conv blocks (two 3×3 convolutions each, channels $b, 2b, \dots$ capped at $8b$, max pooling), global average pooling, dropout and a 4-way linear head, trained with multiclass cross-entropy on *exactly class-balanced* batches (a custom batch sampler draws $B/4$ samples of every condition; labels come from the run-time pipeline).

Specialists. Three autoencoders of the Task 1 architecture are trained independently, each only on one corruption type (salt-and-pepper, blur, occlusion) with the same clean target and the $L_1 + \text{SSIM}$ loss. As allowed, one shared Optuna search

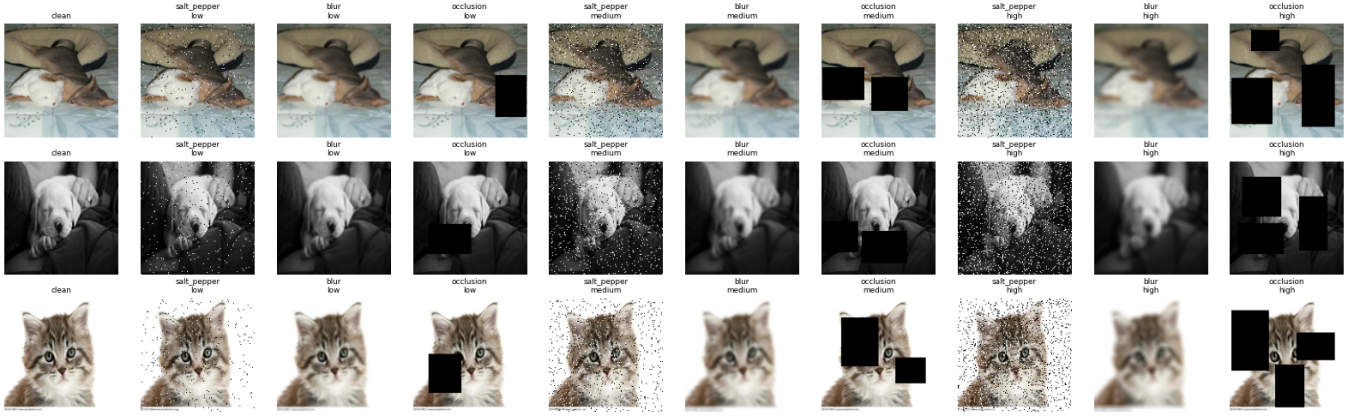


Fig. 1. Test-manifest corruptions for three images (columns: clean, then salt-and-pepper, blur and occlusion at low/medium/high severity).

selects a common architecture (learning rate, bottleneck, channels, batch size, α): every trial trains the three specialists briefly and the objective is their mean validation objective.

Inference. With $r = \arg \max_k p_k$, the output is the identity for $r = \text{clean}$ (bypass, no expert is run) and $A_r(\tilde{x})$ otherwise. We evaluate two modes: *oracle routing* (true label from the manifest) and *predicted routing* (classifier decision).

D. Task 3: jointly trained soft mixture-of-experts

The gate is the Task 2 classifier; the experts are the three trained specialists; an identity branch handles clean inputs. With gate logits $G(\tilde{x})$ and temperature τ , $w = \text{softmax}(G(\tilde{x})/\tau)$ and $\hat{x} = w_0\tilde{x} + w_1A_{s\&p}(\tilde{x}) + w_2A_{\text{blur}}(\tilde{x}) + w_3A_{\text{occ}}(\tilde{x})$, which is differentiable in both gate and experts. Training starts from the Task 2 weights (no random initialisation): first a warm-up stage with frozen experts trains only the gate, then all components are fine-tuned jointly with a smaller learning rate. The loss is

$$\mathcal{L} = \lambda_1 L_1 + \lambda_2 (1 - \text{SSIM}) + \lambda_3 \text{CE}\left(\frac{G(\tilde{x})}{\tau}, y\right) + \lambda_4 \sum_{k=0}^3 \left(\bar{w}_k - \frac{1}{4}\right)^2, \quad (1)$$

with $\lambda_1 = \alpha$, $\lambda_2 = 1 - \alpha$, $\bar{w}_k$ the mean weight of branch k over a class-balanced batch. The cross-entropy keeps the gate tied to the known run-time label (applied to the tempered logits so that it matches the routing weights actually used), and the squared-deviation balance term is a differentiable load-balancing regulariser in the spirit of [8], [9]. Optuna tunes the joint learning rate, τ , λ_3 , λ_4 and α ; a trial is pruned when its intermediate objective is poor *or* when routing collapses (a branch with mean validation weight below 0.02 or above 0.85). The complete pipeline (gate, experts, blending) is exported as one ONNX graph.

E. Task 4: style-conditioned face-to-sketch cGAN

Generator. A U-Net with seven stride-2 stages ($128 \rightarrow 1$) and a mirrored decoder with skip connections (dropout in the first three decoder stages). The style $s \in \{1, 2, 3\}$ is a *learned embedding* of dimension e that is (i) tiled over the image and

concatenated to the input photograph and (ii) concatenated to the 1×1 bottleneck, so it influences both low-level and global features.

Discriminator. A PatchGAN that receives the concatenation of photograph, (real or generated) sketch and the tiled embedding of the same style from its own learned embedding table, and outputs a 14×14 map of real/fake logits.

Losses. $\mathcal{L}_D = \frac{1}{2} [\text{BCE}(D(x, y, s), 1) + \text{BCE}(D(x, G(x, s), s), 0)]$ and $\mathcal{L}_G = \text{BCE}(D(x, G(x, s), s), 1) + \lambda_{L1} \|y - G(x, s)\|_1$. The discriminator real loss, discriminator fake loss, generator adversarial loss and generator L_1 loss are logged separately every epoch, together with validation L_1 /PSNR/SSIM; the same six validation photographs are rendered at fixed intervals to monitor generator development. Optuna tunes the generator and discriminator learning rates, batch size, base channel count, dropout, style-embedding dimension and λ_{L1} on shortened runs; the selected configuration is then retrained for the full schedule.

V. APPLICATION AND DEPLOYMENT

Design. The interface was designed first in Google Stitch (Fig. 2); the React/Tailwind frontend follows its palette (indigo/cyan/teal on dark), fonts (Space Grotesk, Geist) and layout (left navigation with four workspaces, control card, image-panel grid, statistics card).

Architecture. A React + Tailwind single-page application (served by nginx in its own container) calls a FastAPI backend (second container). The backend validates uploads (size limit, real-image check, allowed formats), resizes to 128×128 , optionally applies the selected corruption with the *same* algorithms as training (a NumPy/OpenCV re-implementation that is unit-tested against the training code), runs the ONNX models with ONNX Runtime, and returns base64 PNGs, routing information and timings. Endpoints: GET /api/health, GET /api/samples, POST /api/universal, /api/hard-route, /api/soft-moe and /api/sketch. The four workspaces are: *Universal Restoration* (input, restored output, error map,

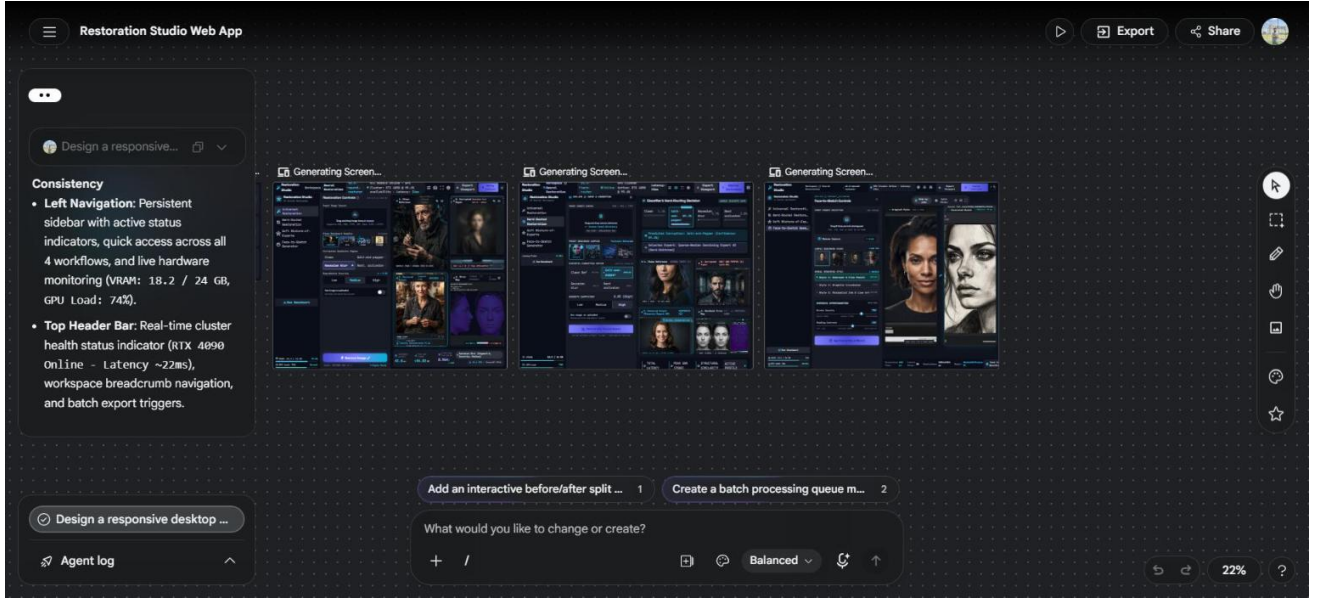


Fig. 2. Original Google Stitch design of “Restoration Studio” (design system and workspace screens) from which the React frontend was implemented.

corruption settings, inference time), *Hard-Routed Restoration* (four classifier probabilities, predicted corruption, selected expert, output, timing), *Soft Mixture-of-Experts* (four routing weights with the strongest contributors highlighted, output, timing) and *Face-to-Sketch Generator* (upload or webcam, Style 1/2/3, side-by-side photo/sketch, download). docker compose up -build starts both containers; the ONNX models are mounted from ./models. Screenshots of the running application are in Section VII.

VI. EXPERIMENTS AND RESULTS

All numbers in this section are measured on the *official test sets* (Pet test: 3,669 images $\times$ 10 conditions = 36,690 manifest entries; FS2K test: 1,046 pairs) with the final selected models, and are produced by the evaluation scripts in the repository; the tables are generated directly from the result files (results/). Hyper-parameter searches used reduced budgets (Section VIII); the final models were retrained for the full schedule with the selected configuration. Training ran on a Colab T4 GPU. “Input” denotes the unrestored corrupted image compared with the clean target (a baseline for how much the model improves the image). PSNR is in dB.

A. Task 1: Universal Autoencoder

Optuna. 24 trials (15 completed, 9 pruned by the median pruner) searched learning rate, batch size, bottleneck type and size, encoder channels, dropout and α (Tables I and II). The first search, which only contained a *dense* (fully-connected) latent vector of 128–1024 numbers, converged to a best validation objective of 0.735 and produced very blurry reconstructions (test: 18.37 dB, SSIM 0.454; Fig. 5). Inspecting those outputs showed that a global fully-connected code discards spatial layout, so a second search added a *spatial* bottleneck (a 1×1 -convolutional latent map of $64 \cdot c_{\text{lat}}$

TABLE I
OPTUNA SEARCH SPACE: TASK 1 UNIVERSAL AUTOENCODER.

Hyper-parameter	Distribution
lr	loguniform[1e-4, 3e-3]
batch_size	categorical[32, 64, 128]
latent_type	categorical[dense, spatial]
latent_dim (dense)	categorical[128, 256, 512, 1024]
latent_ch (spatial; dim = $64 \times \text{latent_ch}$)	categorical[4, 8, 16, 32]
base_ch	categorical[16, 32, 48, 64]
dropout	uniform[0.0, 0.3]
alpha	uniform[0.5, 0.95]

TABLE II
OPTUNA RESULT: TASK 1 UNIVERSAL AUTOENCODER.

Item	Value
lr	0.00090
batch_size	32
latent_type	spatial
base_ch	32
alpha	0.55424
latent_ch	32
dropout	0.07002
trials (complete / pruned / total)	15 / 9 / 24
best trial #, objective	23, 0.5062

numbers that keeps the 8×8 layout; still no skip connections). Optuna selected the spatial type (best objective 0.506) with $c_{\text{lat}} = 32$, i.e. a 2,048-number latent, a $24\times$ compression of the 49,152 input values, and $\alpha = 0.554$ (the initial guess 0.8 was not optimal). The dense baseline is kept in the repository (results/dense_baseline) and used as an ablation.

Results. Over all test conditions the universal autoencoder reaches 21.23 dB / 0.723 SSIM against 22.33 dB / 0.671 for the unrestored input (Table III). Its behaviour depends strongly on the damage: for salt-and-pepper noise it improves the image

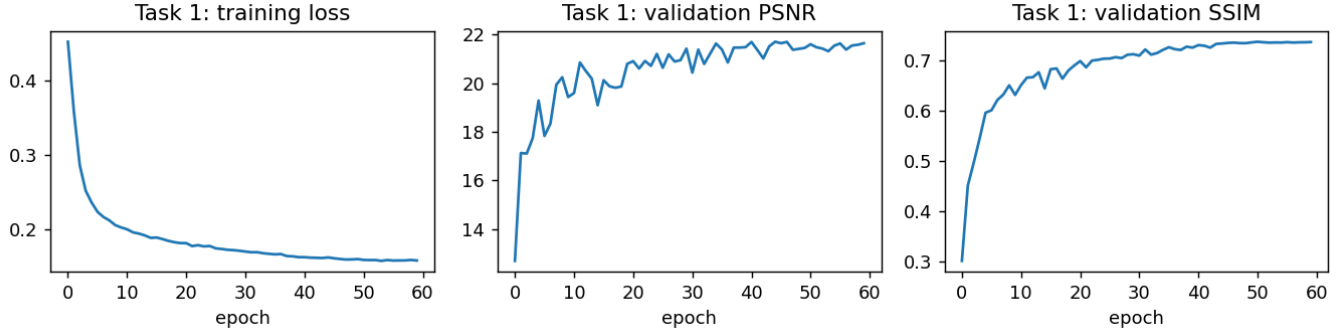


Fig. 3. Task 1 final training: training loss and validation PSNR/SSIM per epoch (from MLflow).

TABLE III

TASK 1 TEST RESULTS (PSNR in dB). INPUT = UNRESTORED CORRUPTED IMAGE.

Condition	PSNR in	PSNR out	SSIM in	SSIM out	n
clean	50.00	21.96	1.000	0.761	3669
salt_pepper / low	20.14	21.95	0.603	0.758	3669
salt_pepper / medium	15.87	21.93	0.339	0.751	3669
salt_pepper / high	13.14	21.84	0.204	0.737	3669
blur / low	32.34	22.06	0.944	0.760	3669
blur / medium	26.77	22.04	0.806	0.753	3669
blur / high	24.35	21.61	0.692	0.711	3669
occlusion / low	16.67	20.71	0.861	0.718	3669
occlusion / medium	13.32	19.74	0.727	0.675	3669
occlusion / high	10.73	18.49	0.537	0.611	3669
overall	22.33	21.23	0.671	0.723	36690

substantially (16.4→21.9 dB, SSIM 0.38→0.75) and the gain grows with severity (13.1→21.8 dB at $p = 0.15$); for occlusion it recovers PSNR (13.6→19.7 dB; 10.7→18.5 dB for 35 % coverage) but SSIM drops slightly (0.709→0.668); for blur and for *clean* inputs it is *worse* than the input (e.g. blur/low 32.3→22.1 dB, clean 50→22.0 dB). This is the cost of the genuine bottleneck: every pixel must be regenerated from a $24\times$ compressed code, so the output quality saturates at about 22 dB irrespective of the input quality (the output PSNR varies by only ≈ 0.1 dB across the three salt-and-pepper severities), whereas inputs with mild damage are already better than that. This observation motivates the identity branch used in Tasks 2 and 3. Fig. 4 shows twelve representative test examples (clean, three severities of each corruption, and two random cases) with the absolute error map. Compared with the dense baseline (Fig. 5) the spatial latent retains object shape, edges and colour, confirming the benefit of the convolutional prior.

Failure cases. Fig. 6 shows the worst (lowest-SSIM) test example for each type. (i) *Clean* (11.8 dB): a pale cat on a pure black background; the network cannot reproduce the sharp black/white transition and leaks a brown halo into the background, an artefact of the bottleneck. (ii) *Salt-and-pepper*, $p = 0.15$ (15.8 dB) and (iii) *blur*, $\sigma = 2.5$ (16.0 dB): the same highly textured meadow image is the worst case for both, because its high-frequency detail is neither present in the blurred input nor representable in the latent. (iv) *Occlusion*,

TABLE IV

OPTUNA SEARCH SPACE: TASK 2 CORRUPTION CLASSIFIER.

Hyper-parameter	Distribution
lr	loguniform[1e-4, 3e-3]
batch_size	categorical[32, 64, 128]
base_ch	categorical[16, 32, 48]
depth	categorical[3, 4, 5]
dropout	uniform[0.0, 0.5]
weight_decay	loguniform[1e-6, 1e-2]

TABLE V

OPTUNA RESULT: TASK 2 CORRUPTION CLASSIFIER.

Item	Value
lr	0.00050
batch_size	32
base_ch	48
depth	5
dropout	0.46691
weight_decay	0.00002
trials (complete / pruned / total)	7 / 13 / 20
best trial #, objective	11, 0.0128

35 % (10.4 dB, SSIM 0.25): the black masks hide most of the cat’s body against a black background; the model fills the hole with a smooth grey region (no semantic inpainting), and this mismatch dominates the error map.

B. Task 2: Classifier and Hard-Routed Specialists

Optuna and training. The classifier search (20 trials, 7 completed, 13 pruned; Tables IV, V) chose five conv blocks with 48 base channels, dropout 0.47 and a small weight decay. The shared specialist search (14 trials, 8 completed; Tables VI, VII) again preferred the spatial bottleneck ($c_{\text{lat}} = 32$, $\alpha = 0.688$). The three specialists were then trained independently, each only on its own corruption.

Classifier. On the 36,690 test entries the classifier reaches accuracy 0.9963, macro-precision 0.9919, macro-recall 0.9959 and macro-F1 0.9939 (Table VIII); the normalised confusion matrix is in Fig. 8. Salt-and-pepper noise is recognised perfectly at all three severities, blur almost perfectly (recall 0.9997; 0.9992 at the weakest setting $k = 3$, $\sigma = 0.7$). The

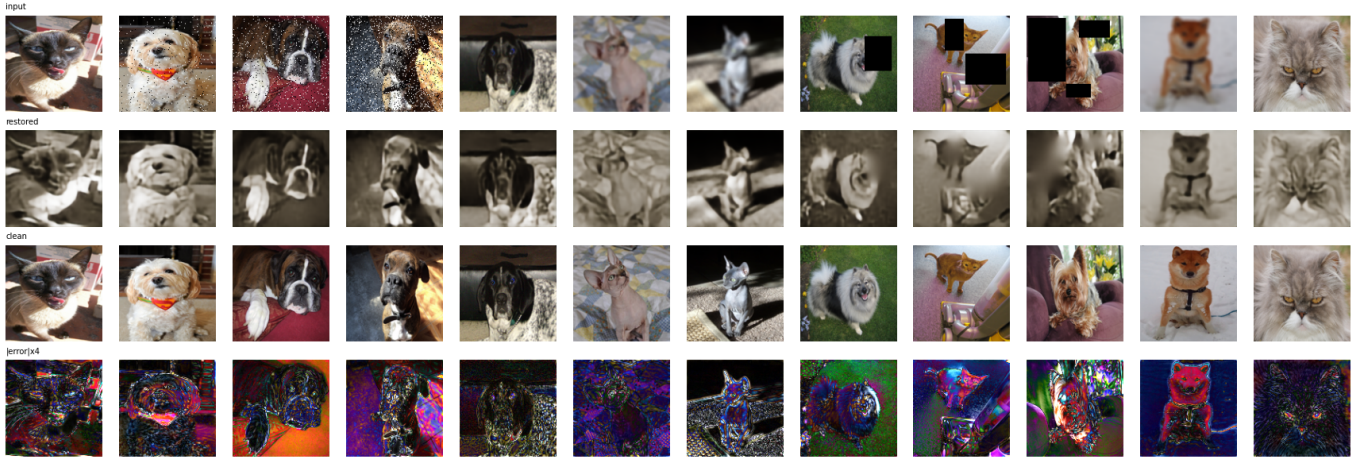


Fig. 4. Task 1, twelve representative test examples (rows: corrupted input, restored output, clean target, $4\times$ absolute error). Columns follow the manifest: clean, salt-and-pepper (low/medium/high), blur (low/medium/high), occlusion (low/medium/high) and two random cases.

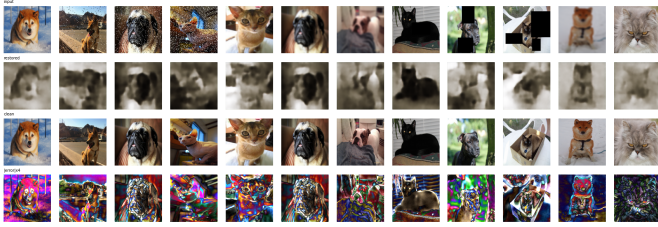


Fig. 5. Ablation: the same test examples reconstructed by the dense-bottleneck autoencoder of the first search (18.37 dB, SSIM 0.454): only coarse colour blobs survive.

TABLE VI
OPTUNA SEARCH SPACE: TASK 2 SPECIALIST AUTOENCODERS (SHARED SEARCH).

Hyper-parameter	Distribution
lr	loguniform[1e-4, 3e-3]
batch_size	categorical32, 64, 128
latent_type	categoricaldense, spatial
latent_dim (dense)	categorical128, 256, 512, 1024
latent_ch (spatial; dim = 64 x latent_ch)	categorical4, 8, 16, 32
base_ch	categorical16, 32, 48, 64
alpha	uniform[0.5, 0.95]
(fixed) dropout	0.1

only noticeable confusion involves the clean class: 1.0% of occluded images are called clean, almost all of them the weakest occlusion (recall 0.9695 for the single 10% rectangle, which hides little), and 0.5% of clean images are called blur; the lowest precision (0.969) belongs to the clean class because it receives these errors, and it has a smaller support (3,669 vs. 11,007).

Restoration with oracle and predicted routing. Table IX gives the two modes. Overall the system reaches 24.79 dB in both modes (SSIM 0.752 oracle, 0.753 predicted), against 22.33 dB / 0.671 for the input and 21.23 dB / 0.723 for the universal autoencoder of Task 1; since the classifier is almost

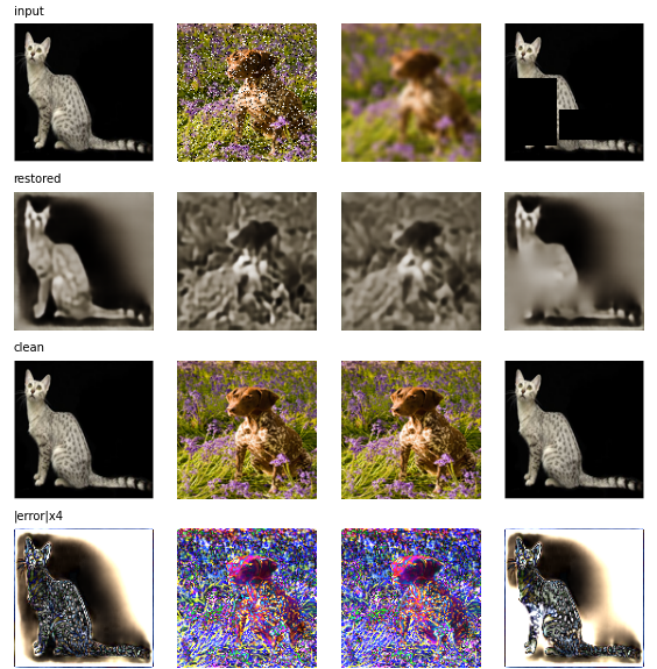


Fig. 6. Task 1 failure cases (worst clean, salt-and-pepper, blur and occlusion test examples).

perfect, predicted routing loses nothing measurable relative to oracle routing. Part of the gain comes simply from the identity bypass: clean images are returned exactly (49.8 dB with the 50 dB cap) while Task 1 damaged them. On *corrupted* images only, the specialists beat the universal model by 1.9 dB (salt-and-pepper: 23.79 vs. 21.90), 0.3 dB (blur: 22.22 vs. 21.91) and 0.3 dB (occlusion: 19.97 vs. 19.65); specialisation helps most when the corruption has a simple, learnable inverse (noise) and little when the missing information cannot be recovered (blur, occlusion). As in Task 1, the blur specialist is below the input PSNR (22.2 vs. 27.8 dB).

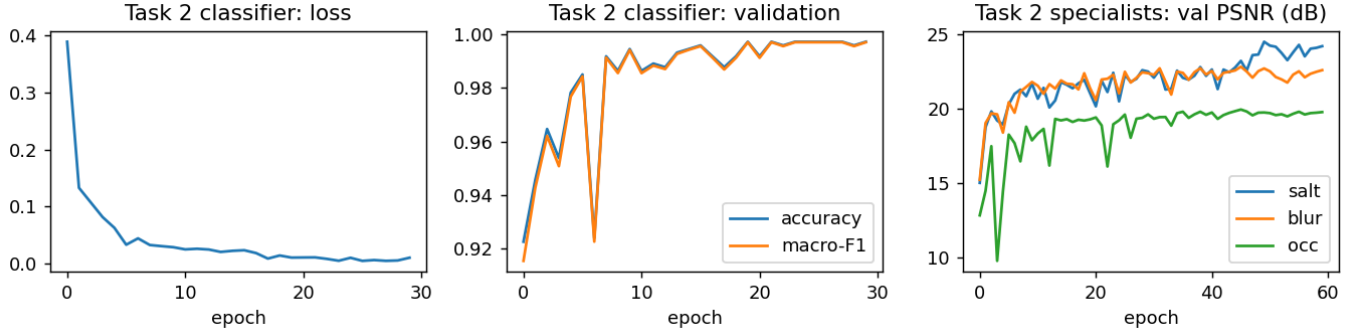


Fig. 7. Task 2: classifier loss and validation accuracy/F1, and validation PSNR of the three specialists.

TABLE VII
OPTUNA RESULT: TASK 2 SPECIALIST AUTOENCODERS (SHARED SEARCH).

Item	Value
lr	0.00156
batch_size	32
latent_type	spatial
base_ch	32
alpha	0.68783
latent_ch	32
trials (complete / pruned / total)	8 / 6 / 14
best trial #, objective	8, 0.5229

TABLE VIII
TASK 2 CLASSIFIER, PER-CLASS TEST METRICS.

Class	Precision	Recall	F1	Support
clean	0.969	0.994	0.982	3669
salt_pepper	1.000	1.000	1.000	11007
blur	0.998	1.000	0.999	11007
occlusion	1.000	0.990	0.995	11007
macro avg	0.992	0.996	0.994	
accuracy			0.996	

Routing failures. 137 of 36,690 test entries (0.37 %) were misrouted. The damaging cases are not those where a corrupted image is sent to the wrong expert but those where a *clean* image is classified as corrupted: the selected expert then regenerates the image from its latent code and destroys its fidelity. Fig. 9 shows the four worst: clean images routed to the blur expert (50dB $\rightarrow$ 10.8, 11.9 and 12.0dB) or to the occlusion expert (50 $\rightarrow$ 13.6dB). Misrouted images lost on average 2.3 dB relative to oracle routing; this is the weakness of hard routing that Task 3 addresses.

C. Task 3: Soft Mixture-of-Experts

Optuna. The 15-trial study (Tables X, XI) tuned the joint learning rate, temperature, λ_{CE} , λ_{bal} and α ; trials with a collapsed or dominating branch would have been pruned, but none of the 15 trials collapsed. The selected configuration has $\tau = 1.43$ (a slightly softened gate), $\lambda_{CE} = 0.019$, $\lambda_{bal} = 0.0074$ and $\alpha = 0.725$. The training used 2 warm-

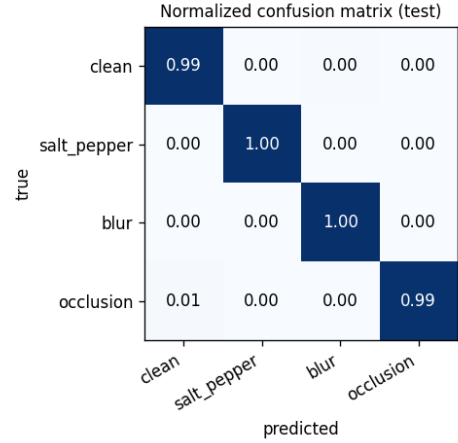


Fig. 8. Task 2: normalised four-class confusion matrix on the test manifest.

TABLE IX
TASK 2 TEST RESTORATION WITH ORACLE VS. PREDICTED ROUTING.

Condition	PSNR oracle	SSIM oracle	PSNR pred.	SSIM pred.
clean	50.00	1.000	49.83	0.999
salt_pepper / low	23.82	0.769	23.82	0.769
salt_pepper / medium	23.80	0.767	23.80	0.767
salt_pepper / high	23.75	0.763	23.75	0.763
blur / low	22.28	0.761	22.29	0.761
blur / medium	22.35	0.759	22.35	0.759
blur / high	22.03	0.729	22.03	0.729
occlusion / low	20.94	0.701	21.02	0.707
occlusion / medium	20.07	0.665	20.07	0.665
occlusion / high	18.90	0.608	18.90	0.608
overall	24.79	0.752	24.79	0.753

up epochs (gate only, learning rate $3\times$ the joint one) followed by 18 joint epochs (Fig. 10); on the validation set the system started at 28.7 dB (right after initialisation from Task 2, where the identity branch already helps clean images) and finished at 30.2 dB.

Restoration. Table XII reports 26.28 dB / 0.814 SSIM overall, which is higher than hard routing (24.79 dB / 0.753), the universal autoencoder (21.23 dB / 0.723) and the input (22.33 dB / 0.671); the corresponding model built on the dense

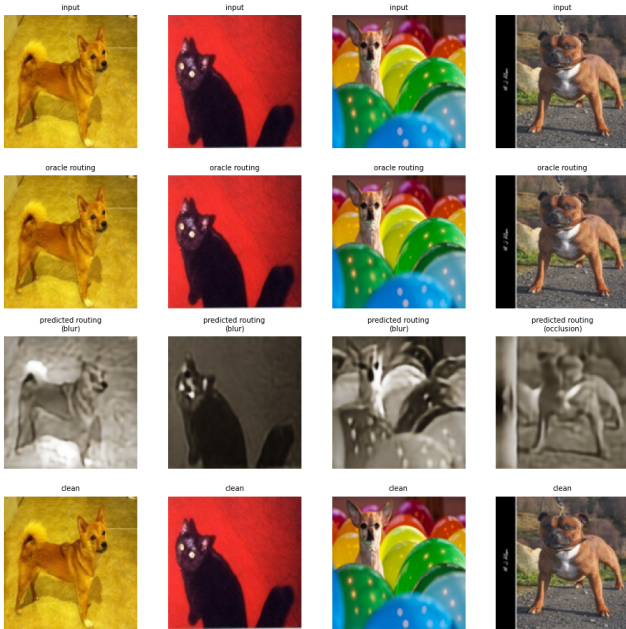


Fig. 9. Task 2 routing failures: clean test images sent to a restoration expert by the classifier (rows: input, oracle routing = identity, predicted routing, clean).

TABLE X
OPTUNA SEARCH SPACE: TASK 3 JOINT FINE-TUNING.

Hyper-parameter	Distribution
lr	loguniform[1e-5, 1e-3]
tau	loguniform[0.5, 2.0]
lambda_ce	loguniform[0.01, 1.0]
lambda_bal	loguniform[1e-3, 0.1]
alpha (L1 weight; SSIM weight = 1 - alpha)	uniform[0.5, 0.95]

bottleneck reached 24.40 dB / 0.723. In particular, the soft blend closes the gap on blur: SSIM 0.827 is *above* the input’s 0.814 and PSNR is 26.4 dB for blur (29.1 dB at the lowest severity, versus 22.1 for the specialist alone), because the gate mixes the input with the expert’s output instead of replacing it. Clean images stay at 49.8 dB, and salt-and-pepper (24.1 dB) and occlusion (20.6 dB, SSIM 0.77) improve on Task 2 as well.

Gate behaviour. Table XIII and Fig. 11 list the mean weights per true condition and severity. The gate has *not* remained a pure classifier (argmax-routing accuracy 0.673, compared with 99.6% for the initial classifier) because the reconstruction loss dominates the small classification weight selected by Optuna; instead it learned a severity-adaptive blending policy: clean $\rightarrow$ identity (0.99); salt-and-pepper $\rightarrow$ salt expert with weight rising from 0.84 (low) to 0.98 (high); blur $\rightarrow$ roughly equal identity and blur expert (0.71/0.28 at low, 0.48/0.52 at high); occlusion $\rightarrow$ identity plus occlusion expert (0.42 at low, 0.58 at high). Every expert is used only for its own corruption type (their weights for unrelated types are numerically zero), so no expert dominates unrelated inputs and none is inactive: the mean weights per branch are

TABLE XI
OPTUNA RESULT: TASK 3 JOINT FINE-TUNING.

Item	Value
lr	0.00063
tau	1.43144
lambda_ce	0.01899
lambda_bal	0.00742
alpha	0.72496
trials (complete / pruned / total)	15 / 0 / 15
best trial #, objective	13, 0.2044

TABLE XII
TASK 3 SOFT MIXTURE-OF-EXPERTS, TEST RESULTS (PSNR IN DB).

Condition	PSNR in	PSNR out	SSIM in	SSIM out	n
clean	50.00	49.75	1.000	0.999	3669
salt_pepper / low	20.14	24.57	0.603	0.804	3669
salt_pepper / medium	15.87	23.90	0.339	0.777	3669
salt_pepper / high	13.14	23.67	0.204	0.767	3669
blur / low	32.34	29.14	0.944	0.916	3669
blur / medium	26.77	25.65	0.806	0.814	3669
blur / high	24.35	24.27	0.692	0.751	3669
occlusion / low	16.67	22.09	0.861	0.866	3669
occlusion / medium	13.32	20.72	0.727	0.778	3669
occlusion / high	10.73	19.02	0.537	0.671	3669
overall	22.33	26.28	0.671	0.814	36690

0.442/0.278/0.127/0.154 (identity/salt/blur/occlusion), with the blur expert the least used, consistent with Task 2 where blur is the corruption that specialisation helps least. Fig. 12 shows examples where one expert completely dominates (heavy salt-and-pepper noise: weight ≈ 1 on the salt expert) and examples where the weights are spread (lighter noise: about 0.65 identity, 0.25 salt expert). The load-balance term is not violated (no branch below 0.05), but note that “balance” is measured on a class-balanced batch: the identity branch still gets the largest average share.

Failure cases. The four worst cases (Fig. 14) are: a clean image with large black areas for which the gate gives 0.53 weight to the occlusion expert (13.2 dB); the gate confuses real dark regions with occlusion, the soft-MoE analogue of the hard-routing failure but milder because the identity still gets 0.47; a heavily textured salt-and-pepper image (17.5 dB, weight 0.93 on the salt expert) whose texture cannot be regenerated; a strongly blurred image (18.1 dB; weights 0.76 identity, 0.24 blur); and a large 35% occlusion (10.8 dB; 0.59 on the occlusion expert) in which the hole cannot be plausibly inpainted by an autoencoder.

D. Task 4: Style-Conditioned Face-to-Sketch cGAN

Optuna and training. The 12-trial study (6 completed, 6 pruned; Tables XV, XVI) used shortened runs of 10 epochs. It selected $g_{lr} = 5.8 \times 10^{-4}$, $d_{lr} = 6.6 \times 10^{-4}$, batch size 16, 64 base channels, dropout 0.47, a 32-dimensional style embedding and $\lambda_{L1} = 272.5$ (near the upper edge of the searched range [10, 300], indicating that a strong pixel term helps on this small dataset). The selected configuration was retrained for the full 150 epochs; the best validation epoch

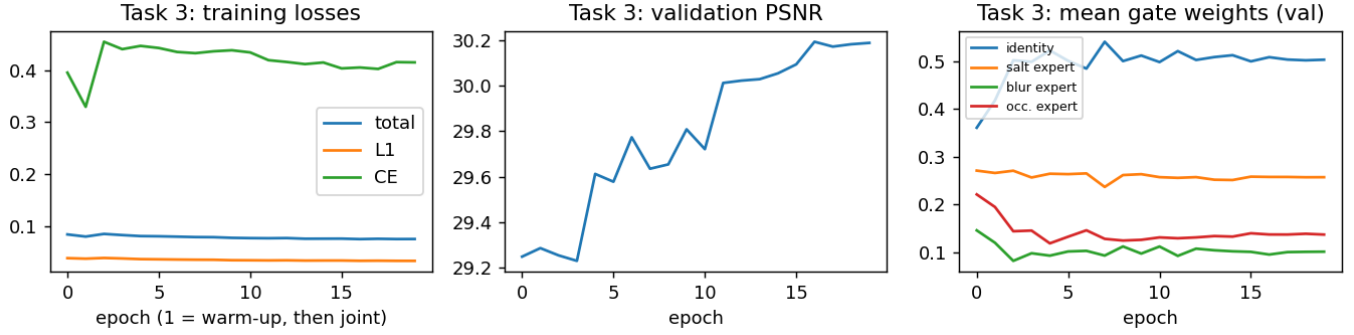


Fig. 10. Task 3 joint training: losses, validation PSNR and mean gate weights per branch.

TABLE XIII
MEAN GATE WEIGHTS PER TRUE CONDITION (TEST).

Condition	identity	salt	blur	occ.
clean	0.995	0.000	0.003	0.003
salt_pepper	0.073	0.925	0.001	0.001
salt_pepper/low	0.153	0.841	0.003	0.003
salt_pepper/medium	0.050	0.949	0.000	0.000
salt_pepper/high	0.016	0.984	0.000	0.000
blur	0.579	0.000	0.420	0.000
blur/low	0.715	0.000	0.285	0.000
blur/medium	0.542	0.000	0.458	0.000
blur/high	0.481	0.000	0.519	0.000
occlusion	0.490	0.000	0.000	0.510
occlusion/low	0.582	0.000	0.000	0.418
occlusion/medium	0.472	0.000	0.000	0.528
occlusion/high	0.415	0.000	0.000	0.585

TABLE XIV
GATE HEALTH ON THE TEST SET.

Statistic	Value
routing accuracy (argmax = true type)	0.673
mean weight per branch	0.442, 0.278, 0.127, 0.154
inactive branches (mean weight < 0.05)	none
mean gate entropy (nats)	0.451

is used. Fig. 15 shows the separately logged discriminator real/fake losses, generator adversarial and L_1 losses and validation metrics; the discriminator stays near 0.35–0.45 (neither side collapses) with an isolated instability near epoch 145. The training L_1 is higher than validation L_1 because training uses dropout and random paired crops. Fig. 16 shows the same validation photographs at epochs 1, 10, 50, 100 and 150: from an empty grey output at epoch 1 to recognisable faces at epoch 10 and sharp pencil-like strokes by epoch 50, after which improvements are subtle.

Results. On the official test set the generator reaches $L_1 = 0.102$, 15.9 dB and SSIM 0.493 (Table XVII). Performance differs by style: Style 1 (619 test pairs) 17.8 dB / 0.542, Style 2 (381 pairs) 12.4 dB / 0.397 and Style 3 (only 46 pairs, so a noisy estimate) 19.1 dB / 0.629. Pixel metrics are low in absolute terms because hand-drawn strokes are not pixel-aligned with edges in the photograph, and Style 2

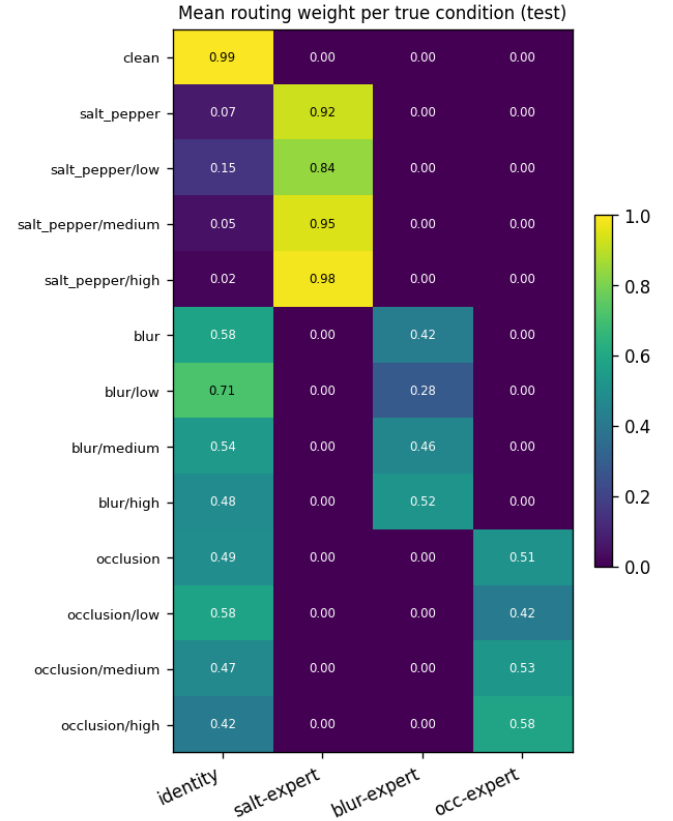


Fig. 11. Task 3: mean routing weight per true corruption type and severity (test).

sketches are dark and heavily hatched, which an L_1 -driven generator reproduces as softer, smoother strokes (Figs. 18, 17); metrics such as FID or LPIPS were not computed. The style embedding is used by both networks, and the conditioning has a clear effect (Fig. 17): for the same photographs, Style 1 yields light, thin contour sketches while Style 2 and Style 3 yield darker, bolder shading.

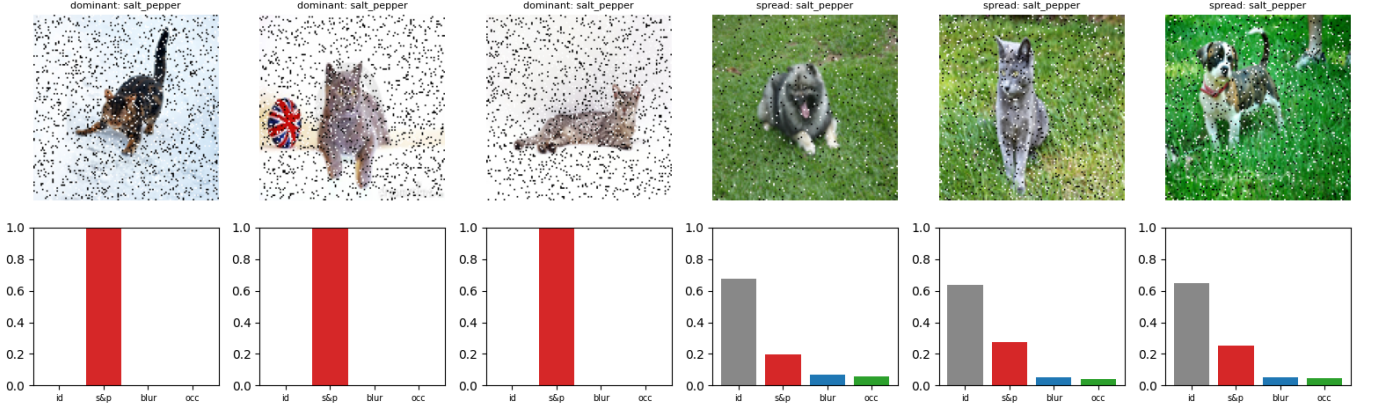


Fig. 12. Task 3 routing weights for individual test images: one expert dominating (left three) and distributed weights (right three). Bars: identity, salt, blur, occlusion.

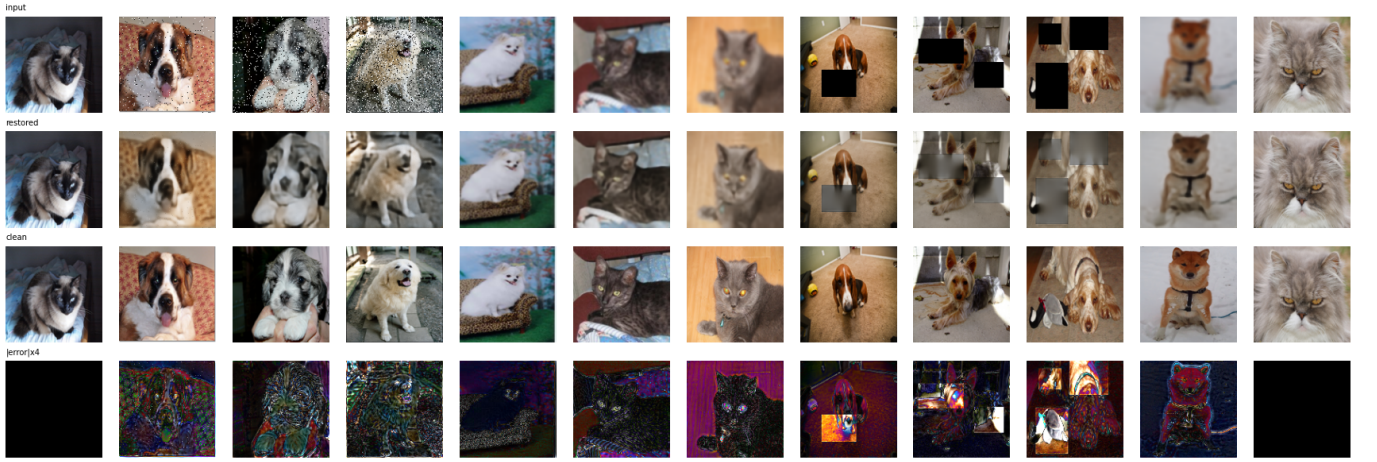


Fig. 13. Task 3 soft-MoE restoration: twelve representative test examples (input, restored, clean, 4× error).

TABLE XV
OPTUNA SEARCH SPACE: TASK 4 FACE-TO-SKETCH CGAN.

Hyper-parameter	Distribution
g_lr	loguniform[5e-5, 1e-3]
d_lr	loguniform[5e-5, 1e-3]
batch_size	categorical8, 16, 32
base_ch	categorical32, 48, 64
dropout	uniform[0.0, 0.5]
style_dim	categorical8, 16, 32, 64
lambda_l1	loguniform[10, 300]

TABLE XVI
OPTUNA RESULT: TASK 4 FACE-TO-SKETCH CGAN.

Item	Value
g_lr	0.00058
d_lr	0.00066
batch_size	16
base_ch	64
dropout	0.47145
style_dim	32
lambda_l1	272.54610
trials (complete / pruned / total)	6 / 6 / 12
best trial #, objective	9, 0.6572

E. ONNX Export and Consistency

All seven inference models (universal autoencoder, classifier, three specialists, soft-MoE pipeline and sketch generator) were exported to ONNX (opset 17, dynamic batch) and run with ONNX Runtime on real test inputs (deterministically corrupted test images and FS2K photographs). Table XVIII shows that the maximum absolute difference to PyTorch is below 4×10^{-6} , far below the 10^{-4} threshold, for every model. On the development laptop’s CPU (Intel i5-7200U) the

application measured about 25 ms per image for the universal autoencoder, about 104 ms for the full soft-MoE pipeline and 67 ms for the sketch generator.

F. Experiment Tracking

Every training run and every Optuna trial was logged to MLflow (parameters, per-epoch metrics, validation samples and checkpoints; the SQLite database is in `experiments/`).

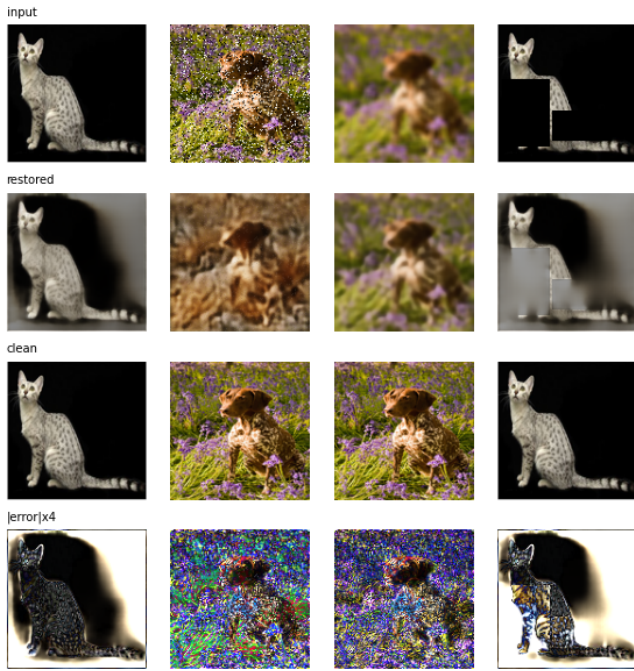


Fig. 14. Task 3 failure cases (worst clean, salt-and-pepper, blur and occlusion).

TABLE XVII
TASK 4 TEST RESULTS ON THE OFFICIAL FS2K TEST SET (IMAGES IN [0,1]).

Style	L1	PSNR (dB)	SSIM	n
all	0.1023	15.91	0.493	1046
Style 1	0.0747	17.84	0.542	619
Style 2	0.1522	12.40	0.397	381
Style 3	0.0611	19.09	0.629	46

Fig. 19 shows the MLflow views of the Task 3 run table and the Task 4 metric charts. Figures of the Optuna optimisation histories (universal autoencoder, soft MoE and GAN) are in Fig. 20.

VII. APPLICATION SCREENSHOTS

Figs. 21–24 show the four workspaces of the running application with the final ONNX models (screenshots taken with the same Docker-independent dev stack; the Docker Compose stack serves the identical frontend and backend). The Universal and Hard-Routed pages show the clean reference, corrupted input, restored output and error map, the selected corruption settings and the inference time; the Hard-Routed page additionally shows the four classifier probabilities, the predicted corruption and the selected expert; the Soft-MoE page shows the four routing weights with the strongest contributors; the Face-to-Sketch page supports upload or webcam, the three style conditions, side-by-side display and a download button.

TABLE XVIII
PYTORCH VS. ONNX RUNTIME CONSISTENCY (MAX ABS. DIFFERENCE, REAL TEST IMAGES).

Model	max ldiffl	passed ($< 10^{-4}$)	size (MB)
universal_ae	7.15e-07	yes	9.6
classifier	1.19e-07	yes	21.2
specialist_salt	8.05e-07	yes	9.6
specialist_blur	7.00e-07	yes	9.6
specialist_occ	5.36e-07	yes	9.6
soft_moe	4.77e-07	yes	50.0
sketch_generator	3.87e-06	yes	168.5

VIII. LIMITATIONS

- **Reduced search budgets.** Optuna trials used 6–8 epochs on a 1,500-image subset (10 epochs for the GAN) and 12–24 trials per task; the CPU/GPU budget (a free Colab T4 and a one-day deadline) did not allow larger studies. λ_{L1} for the GAN landed near the edge of its range and the autoencoder objective also favoured the largest searched latent size, so wider ranges could improve results. Each configuration was run with a single seed, so small differences (e.g. oracle vs. predicted routing, 0.001 SSIM) are not statistically established.
- **Bottlenecked reconstruction.** With a genuine bottleneck and no skip connections the restoration autoencoders cannot match the fidelity of mildly damaged inputs (blur, low-severity noise, clean). The identity branch of Tasks 2 and 3 compensates, but the specialists remain weaker than the input in that regime; a limited skip connection (allowed by the assignment if justified) was not investigated.
- **Soft-MoE interpretability.** The tuned classification weight is small, so the gate acts as a severity-adaptive blender rather than a classifier (argmax routing 0.673); a larger λ_{CE} would make weights more class-like at some cost in quality (we did not tune this trade-off with a routing-aware objective). Compound corruptions (e.g. blur + noise) were not evaluated.
- **GAN evaluation.** Only PSNR/SSIM/ L_1 were used (no FID/LPIPS or user study); Style 3 has just 46 test pairs; FS2K contains few demographics; training images are only 128×128 .
- **Deployment.** Models run on CPU through ONNX Runtime; the demo corrupts uploaded images with the same synthetic operators as in training, so genuinely different real-world corruptions may behave differently. The Docker images were built and smoke-tested in continuous integration on GitHub; the models are supplied separately (download link in the README).

IX. CONCLUSION

We built and compared three restoration strategies on a common data protocol. A universal convolutional autoencoder with a genuine latent bottleneck restores severe noise and occlusion but cannot beat mildly damaged inputs; a bottleneck that keeps spatial layout is essential (18.4→21.2 dB over a dense latent). Hard routing by a 99.6%-accurate classifier

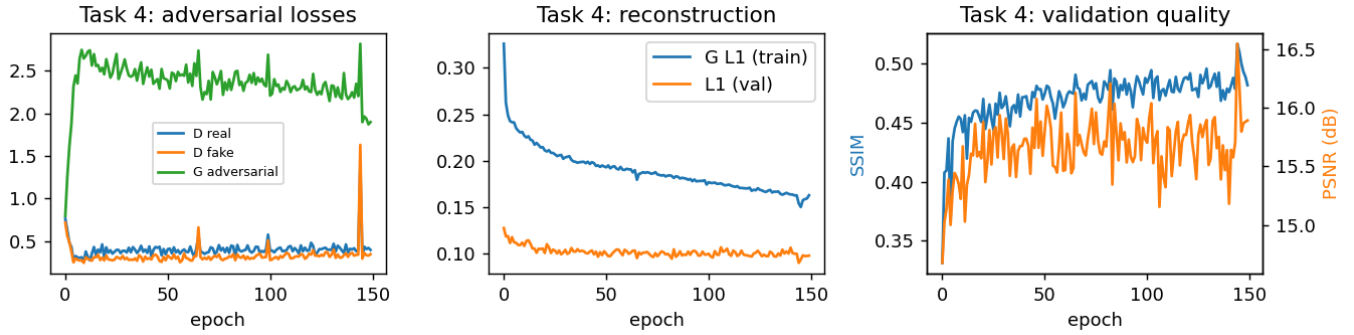


Fig. 15. Task 4 training: discriminator real/fake loss, generator adversarial loss, generator L_1 (train/val) and validation PSNR/SSIM.

with specialists and an identity bypass raises the test PSNR to 24.8dB with predicted routing equal to oracle routing, but its rare errors on clean images are destructive. A jointly fine-tuned soft MoE reaches 26.3 dB / 0.814 SSIM by blending identity and experts according to severity, at the price of gate weights that no longer mirror the corruption label. A style-conditioned pix2pix generator with a learned style embedding produces recognisable, style-dependent sketches (test SSIM 0.49), and all four systems were tuned with Optuna, tracked with MLflow, verified against ONNX and deployed in a single web application whose Docker Compose configuration starts the complete stack.

APPENDIX A AI-USE STATEMENT

Anthropic’s Claude (via Claude Code and the Claude desktop application) was used for: drafting and refactoring the Python code (data pipeline, models, training/Optuna/evaluation scripts, FastAPI backend, React frontend, Docker files, CI workflow), operating the Colab notebook and Google Stitch through the browser, and drafting this report. Google Stitch generated the first interface design (Fig. 2). Every component was tested rather than trusted: unit tests for the split, corruption statistics, manifest determinism, SSIM (against scikit-image), model shapes, GAN conditioning and paired augmentation, and the API (including a test that the backend’s corruptions equal the training code’s); PyTorch-vs-ONNX numerical comparison; a GitHub Actions workflow that builds and starts the Docker stack; and end-to-end runs of the application on real models. Errors found and corrected during development included a .gitignore rule that hid the data package, SQLite/MLflow stalls on a network drive, a half-precision dtype mismatch in the hard router (found when the first Task 2 evaluation crashed), upper-case FS2K file extensions that break on Linux, and the initial dense-bottleneck design whose blurry results prompted the spatial-latent investigation. All numerical results reported here are copied by script from the files produced by the final runs. The author is responsible for the submitted work and can explain and modify any component.

APPENDIX B REPRODUCTION

Repository: <https://github.com/MusaxKhan/Restoration-GAN>. The README gives the Docker commands (python scripts/download_models.py, then docker compose up -build), the data-preparation, training, Optuna, evaluation and ONNX-export commands (notebooks/colab_training.ipynb, scripts/run_v2.sh, scripts/finish_v2.sh), and the model download link. Experiment records (MLflow database, Optuna studies, best configurations) are under experiments/ and configs/.

REFERENCES

- [1] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in *Proc. ICML*, 2008, pp. 1096–1103.
- [2] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 2006.
- [3] O. Ronneberger, P. Fischer, and T. Brox, “U-net: convolutional networks for biomedical image segmentation,” in *Proc. MICCAI*, 2015, pp. 234–241.
- [4] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, “Loss functions for image restoration with neural networks,” *IEEE Trans. Comput. Imaging*, vol. 3, no. 1, pp. 47–57, 2017.
- [5] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [6] B. Li, X. Liu, P. Hu, Z. Wu, J. Lv, and X. Peng, “All-in-one image restoration for unknown corruption,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 17 452–17 462.
- [7] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [8] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: the sparsely-gated mixture-of-experts layer,” in *Proc. ICLR*, 2017.
- [9] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: scaling to trillion parameter models with simple and efficient sparsity,” *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.
- [10] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Proc. NeurIPS*, 2014.
- [11] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” *arXiv:1411.1784*, 2014.
- [12] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *Proc. IEEE CVPR*, 2017, pp. 1125–1134.



Fig. 16. Generator development on fixed validation photographs (columns: photograph, generated sketch at epochs 1, 10, 50, 100, 150, ground-truth sketch).

- [13] C. Li and M. Wand, “Precomputed real-time texture synthesis with Markovian generative adversarial networks,” in *Proc. ECCV*, 2016, pp. 702–716.
- [14] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. V. Gool, “Facial-sketch synthesis: a new challenge,” *Mach. Intell. Res.*, vol. 19, pp. 257–287, 2022.
- [15] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: a next-generation hyperparameter optimization framework,” in *Proc. ACM SIGKDD*, 2019, pp. 2623–2631.
- [16] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi *et al.*, “Accelerating the machine learning lifecycle with MLflow,” *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.
- [17] ONNX Runtime developers, “ONNX Runtime: cross-platform, high performance ml inferencing,” <https://onnxruntime.ai>, 2024.
- [18] S. Ramírez, “FastAPI framework,” <https://fastapi.tiangolo.com>, 2024.
- [19] Google, “Google Stitch: design with AI,” <https://stitch.withgoogle.com>, 2025.
- [20] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, “Cats and dogs,” in *Proc. IEEE CVPR*, 2012, pp. 3498–3505.
- [21] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.
- [22] L. N. Smith and N. Topin, “Super-convergence: very fast training of neural networks using large learning rates,” *Proc. SPIE Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, 2019.
- [23] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [24] S. Ioffe and C. Szegedy, “Batch normalization: accelerating deep network training by reducing internal covariate shift,” in *Proc. ICML*, 2015, pp. 448–456.

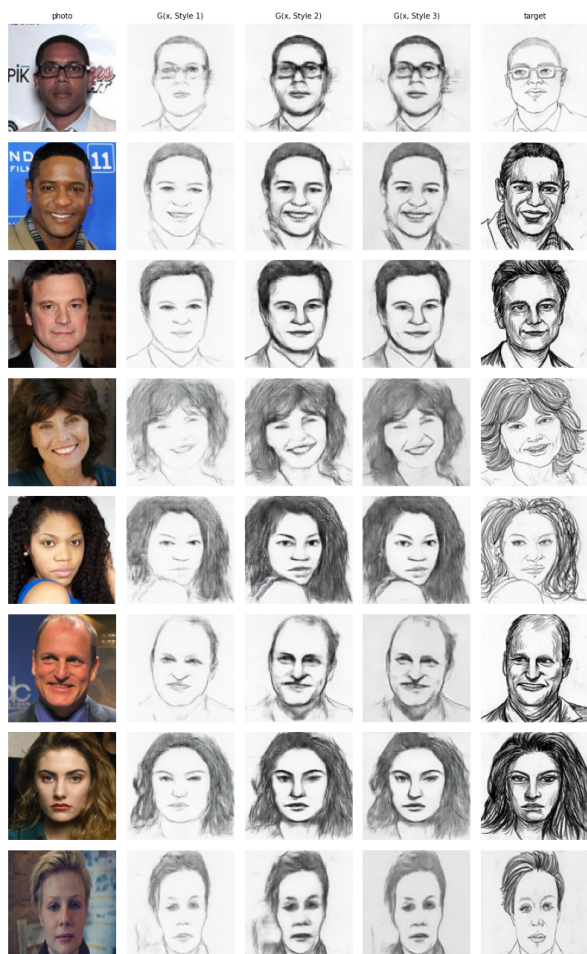


Fig. 17. Conditioning check on test photographs: $G(x, \text{Style } 1/2/3)$ and the ground-truth sketch.

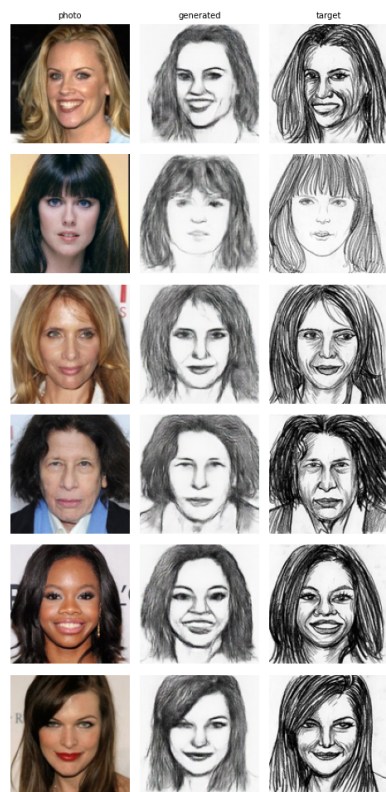


Fig. 18. Lowest-SSIM test examples: the generated sketches are plausible but the strokes differ from the artist's dense hatching, which pixel-wise metrics penalise heavily.

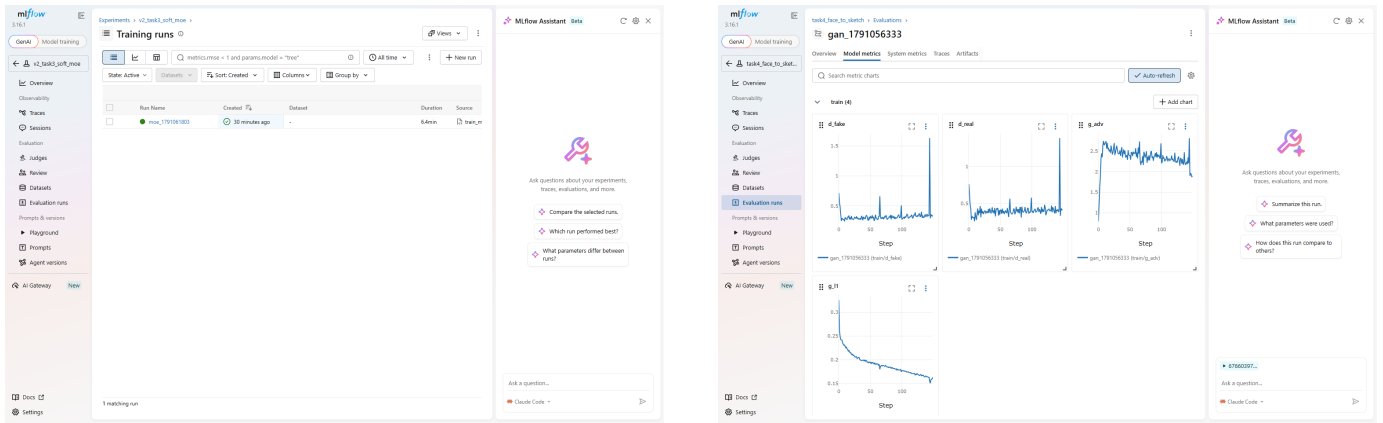


Fig. 19. MLflow tracking UI: Task 3 experiment run table (left) and Task 4 training metric charts (right).

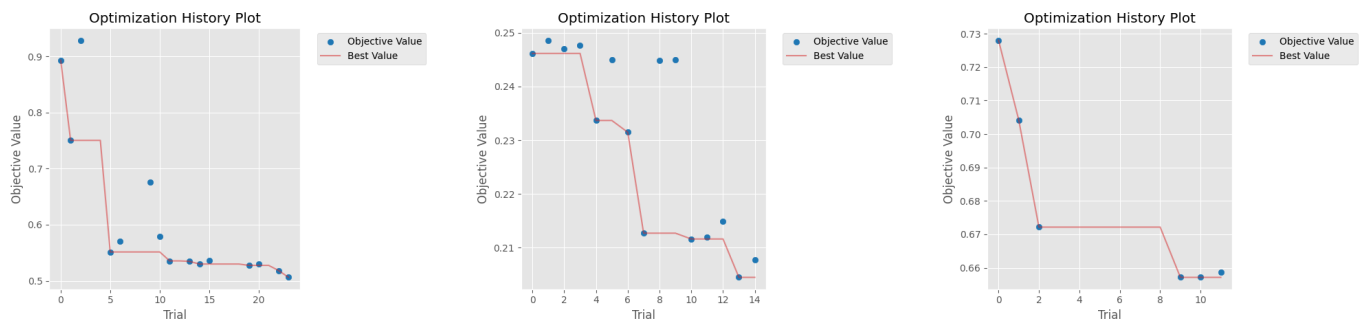


Fig. 20. Optuna optimisation histories: Task 1 universal autoencoder, Task 3 soft MoE, Task 4 cGAN.

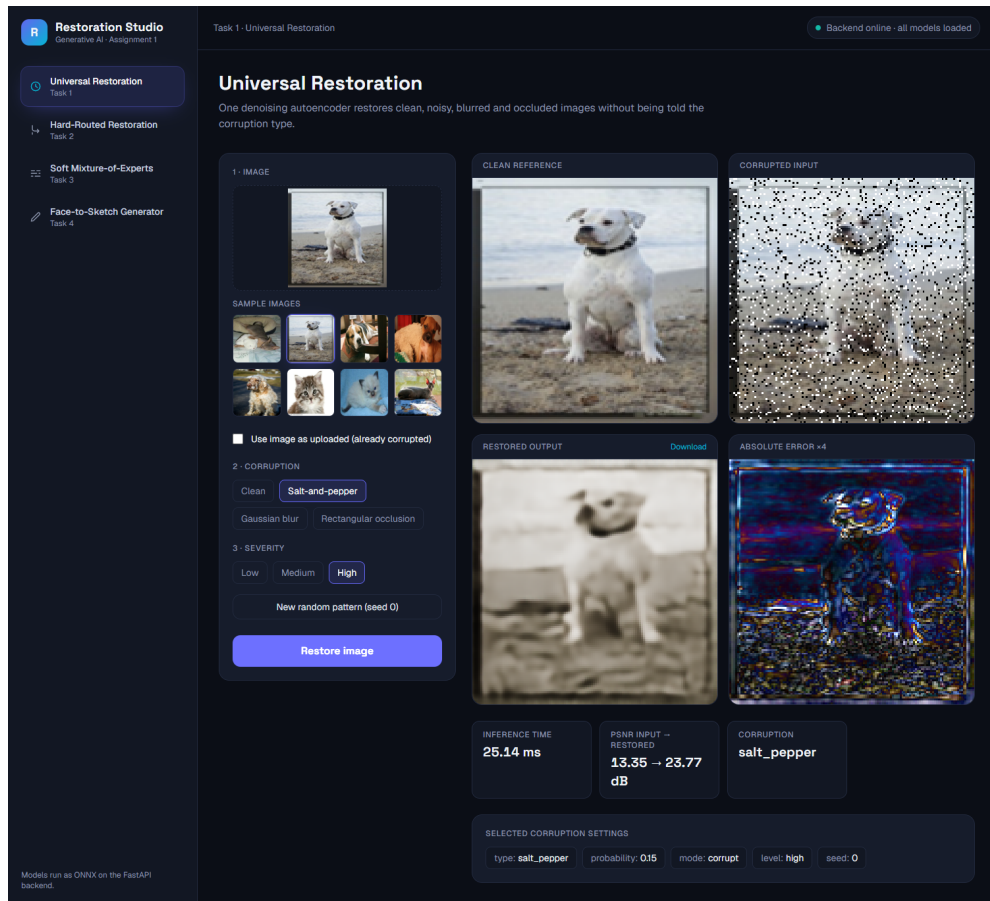


Fig. 21. Universal Restoration workspace (salt-and-pepper, high severity).

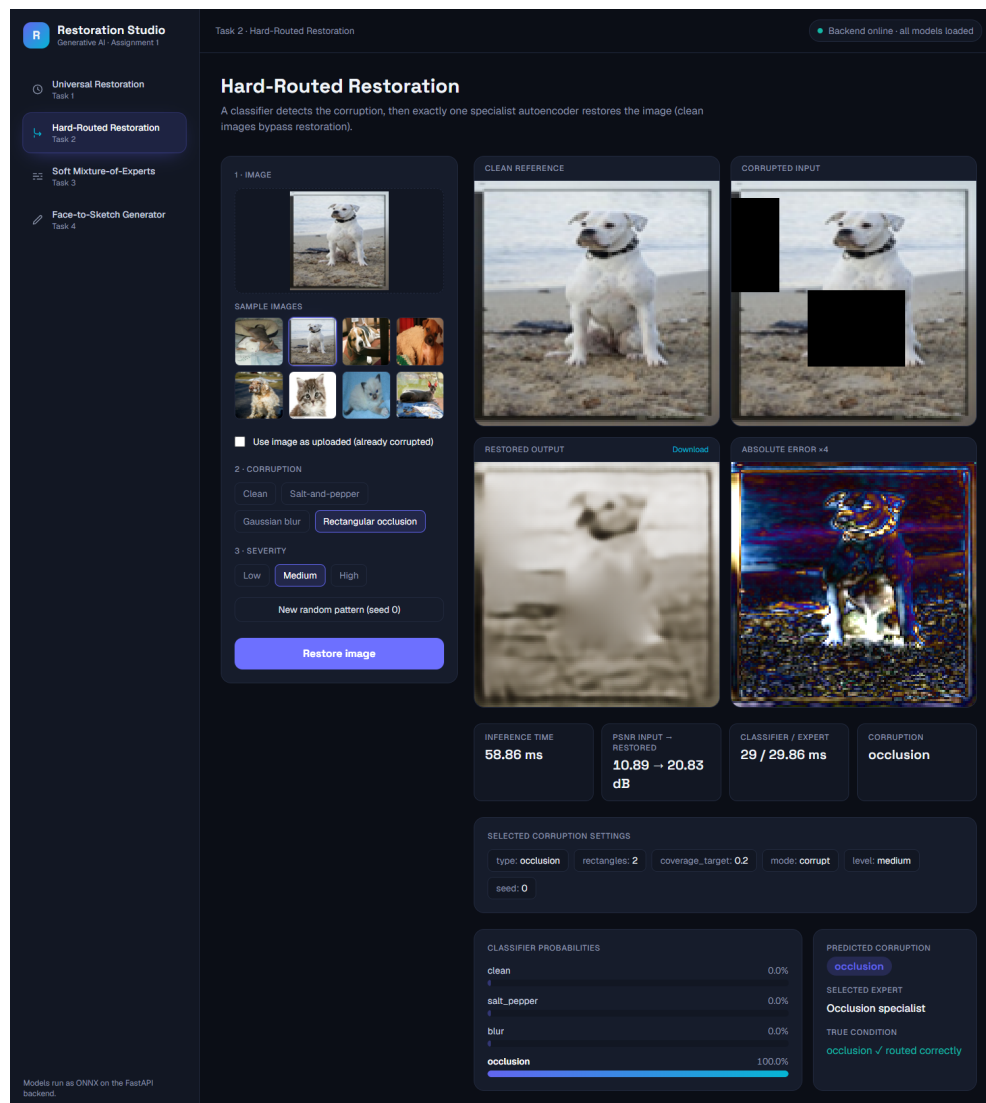


Fig. 22. Hard-Routed Restoration workspace: classifier probabilities, predicted corruption and selected expert.

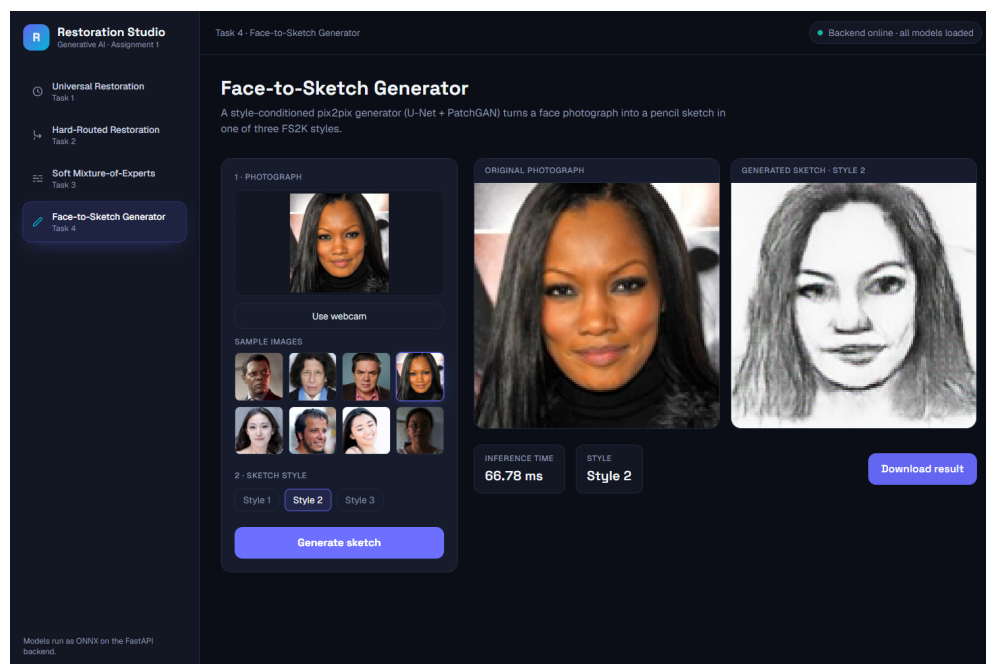


Fig. 24. Face-to-Sketch Generator workspace (Style 2) with side-by-side result and download.